

NG Feedgas Production Tracker — Masterplan & Layout

Onboarding / handoff document. Read this before building on top of the project.

1. What this project is (the mental model)

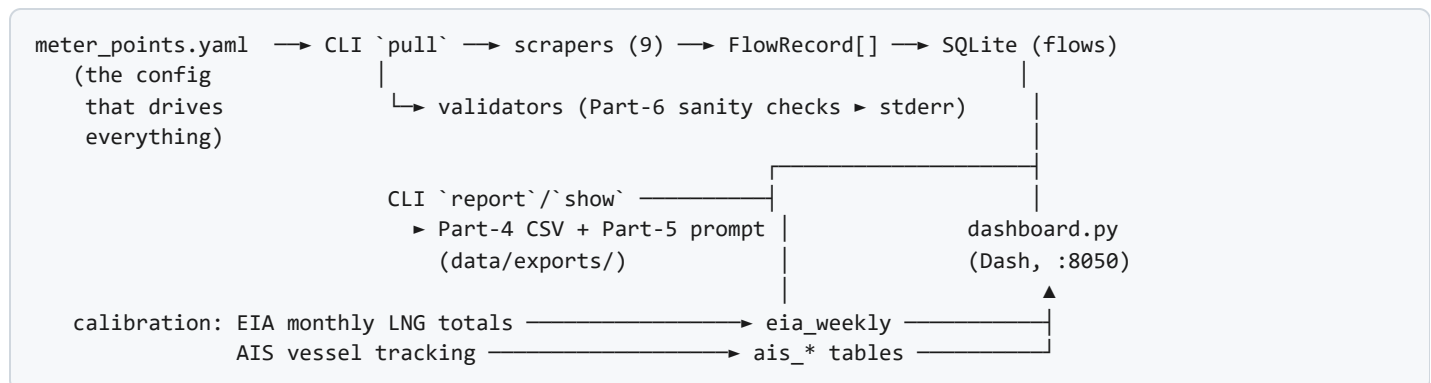
It automates a manual analyst workflow defined in `NG Production/LNG_Feedgas_Template.md`: every gas day, pull scheduled-quantity data from public pipeline **EBB portals** (Electronic Bulletin Boards — FERC-mandated capacity postings), sum the flows per terminal/border point, store the history, and emit reports + a dashboard.

The core idea: **every scraper, no matter how different the portal, produces the same canonical record** — a `FlowRecord` (one metered flow at one point on one pipeline for one gas-day/cycle). Once data is in that shape, all downstream code (storage, CSV export, analysis prompt, validators, dashboard) is scraper-agnostic.

Important framing: The `README.md` and `docs/portal_notes.md` describe an early "LNG-only proof of concept (6 of 8 terminals, ~6,900 MMcf/d)." **The project has since outgrown those docs.** It now also tracks **U.S.→Mexico border exports** and **Canada border crossings** (imports + exports). The live DB shows **~10,365 MMcf/d across 17 terminals for 2026-05-27**. Trust the code and `meter_points.yaml`, not the prose docs.

2. Workflows — how the program actually runs

2.0 The high-level data flow



There are **two independent timelines** here, and it matters that you keep them apart:

- **Write side (scraping):** `pull` is the only thing that writes the `flows` table. It is meant to run once or twice a day (timely + evening cycles) from Task Scheduler. It hits the live internet.
- **Read side (reporting/UI):** `report`, `show`, and the dashboard only *read* `feedgas.db`. They never scrape. So the dashboard shows whatever the last `pull` captured — if it looks stale, run `pull` again. This separation is deliberate: the slow, failure-prone network work is isolated from the fast, always-available presentation work.

The rest of this section walks each workflow end-to-end, in execution order, with the real function names so you can follow along in the source.

2.1 `pull` — the daily scrape (the heart of the write side)

Entry point: `pull()` in `src/ng_feedgas/cli.py`. Step by step:

1. **Resolve the gas day.** `_parse_date()` turns `today` / `yesterday` / `YYYY-MM-DD` into a `date`. "Gas day" = the 24-hour delivery day the data describes.
2. **Load the config.** `load_config()` parses `meter_points.yaml` into a `Config` object: a flat list of `MeterPoint`s (one per `feeds:` entry), a `{terminal: nameplate}` map, and the US-total bounds. This config is what *drives* the whole run — the scrapers don't know any meter IDs themselves, they're told which ones to look for.
3. **Decide which scrapers to run.** `_build_scrapers(cfg, only):` - If `--pipeline all`, take every key in the `SCRAPERS` registry; otherwise just the one named. - For each scraper name, look up its class and call `cfg.for_scraper(name)` to get *only the meter points assigned to that scraper* in the YAML. - If a scraper has zero configured meters, it's skipped with a note (e.g. an empty terminal). - Returns a dict of `{scraper_instance: [its meter points]}`.
4. **Open the DB.** `connect()` opens `data/feedgas.db` and runs `schema.sql` (idempotent `CREATE TABLE IF NOT EXISTS`), so the file is created on first run.
5. **Run each scraper in isolation.** For every `(scraper, meters)` pair: - Build a `ScrapeContext(gas_day, cycle, meter_points, request_delay_s)`. - Call `scraper.fetch(ctx)` inside a `try/except` — **a crash in one scraper is logged to stderr and skipped, never aborting the others**. This is why a dead portal doesn't lose you the rest of the day's data. - `upsert_flows(conn, records)` writes the returned `FlowRecord`s.
6. **Validate what landed.** After all scrapers, `terminal_totals(conn, gas_day, cycle)` re-reads the day's totals from the DB and `validate(cfg, totals)` runs the Part-6 checks. Warnings print to **stderr** (so cron/Task-Scheduler logs surface them) but never block the write.
7. **Report the count.** Prints total rows written and the DB path.

Upsert semantics (`upsert_flows` in `storage/db.py`): the insert is `ON CONFLICT (gas_day, cycle, pipeline, meter_point) DO UPDATE`. So re-running `pull` for the same day/cycle **overwrites** those rows rather than duplicating — safe to re-run as a portal refreshes its posting through the day.

2.2 Inside a scraper's `fetch()` — the common shape

Every scraper is different on the *outside* (one portal is a CSV URL, another is a JS SPA behind bot protection) but identical on the *inside*. `fetch(ctx) -> list[FlowRecord]` always does:

1. **Group the meters** by `pipeline_code` if the portal serves several pipelines under one host (e.g. KMI serves NGPL, TGP, EEC, EPNG... each needs a separate download keyed by `code=`).
2. **Fetch the portal's snapshot** for the gas day + cycle. This is the only step that differs: - *Direct CSV* (`energytransfer`, `et_ipost`): GET a landing page for cookies, then GET the `?f=csv` URL. - *ASP.NET viewstate* (`kmi`, `enbridge`): GET the form, scrape `__VIEWSTATE` / `__EVENTVALIDATION` and the download button name, then POST to trigger an Excel/CSV download. `kmi` adds retry-with-backoff and a fresh session per attempt (the KM portal drops connections). - *Server-rendered HTML* (`williams`, `williams_nwp`): multi-step JSP/Struts navigation, then regex the `<TR><TD>` rows out of the report HTML. - *Headless browser* (`tceconnects`, `iroquois`): Playwright drives Chromium to click through a JS-only UI and capture a CSV download or scrape a virtual-scrolling grid.
3. **Match meters to rows** (`_match_meters`): for each `MeterPoint`, find its row — **by `meter_id` first, falling back to a `location_name` substring match**. This two-tier match is why the YAML carries both: the ID is exact, the name is the human-readable fallback if a portal rennumbers.
4. **Parse + normalize** (`_parse_number`): strip commas, pull the number out of the "Total Scheduled Quantity" cell, then **divide by 1000 to convert Dth/d → MMcf/d**.
5. **Resolve direction**: read the portal's "Flow Ind" column (`D / R / BD` or "Delivery"/"Receipt"); if unreadable, fall back to the `direction`: declared in the YAML. `BD` (bi-directional) is treated as `delivery` for feedgas.
6. **Emit a `FlowRecord`** per matched meter, stamped with `source_url` and `scraped_at`.

If a meter isn't found, the scraper logs a warning and moves on — it emits *fewer* records rather than zeros, so a missing match shows up as "no data" downstream, not as a misleading 0 MMcf/d.

2.3 `report` and `show` — emitting Part-4 and Part-5

These are pure read-side commands (no scraping). Both read `feedgas.db` for the requested gas-day/cycle:

- `show` → `render_part4_text()` prints the Part-4 entry table to stdout (terminal → pipeline → meter → MMcf/d, with per-terminal and U.S. totals).
- `report` → writes two files to `data/exports/`:
 - `<day>_<cycle>.csv` — the Part-4 table as CSV (`write_part4_csv`).
 - `<day>_<cycle>.prompt.txt` — the Part-5 analysis prompt (`write_prompt`), pre-filled with the day's totals plus **day-over-day, 7-day, and 30-day averages** computed by `analysis/stats.py`'s `compute()` (which calls `us_totals_range()` for the trailing windows). The "Known outages" line is left as a manual fill-in — there's no automated source for it. The prompt is ready to paste into Claude.

⚠ Both of these use the **hardcoded 8-LNG `TERMINAL_ORDER`** in `storage/export.py`, so Mexico and

Canada points are *not* included in the CSV or prompt. See §8.

2.4 Calibration — the two optional cross-check sources

These don't feed the `flows` table; they populate separate tables the dashboard overlays so you can sanity-check how big your scraping gap is.

- **eia-fetch** (`calibration/eia.py`): hits the EIA v2 API (`natural-gas/move/poe2` , `process=ENG`) for U.S. LNG exports, converts monthly MMcf totals → Bcf/d, and upserts into `eia_weekly` (the table name is historical; the data is now monthly since EIA retired the weekly series). Needs a free `EIA_API_KEY` . The dashboard draws this as a dashed reference line.
- **ais-collect** (`calibration/ais.py`): opens a WebSocket to aisstream.io, subscribes to bounding boxes around the 8 LNG terminals, and for ~N seconds records two message types — `ShipStaticData` (vessel name/type → persisted in `ais_ships` , an MMSI registry) and `PositionReport` (lat/lon/speed/status → `ais_observations`). Needs `AISSTREAM_API_KEY` . LNG carriers moor for 12–24h, so this is run **hourly** to accumulate berth occupancy.
- **ais-infer** (`calibration/ais.py`): rolls a day's observations into one row per terminal in `ais_daily_inference` . The rule is deliberately crude: if any LNG-carrier-type vessel (`ship_type` 70–89) was moored or slow (`sog < 1` or `nav_status = 5`) in the box that day, count it "at berth" and estimate feedgas at **60% of nameplate**; otherwise 0. This is *directional* signal (is Cove Point / Calcasieu loading at all?), not a precise volume.

2.5 The dashboard (`dashboard.py`)

A Plotly Dash app on `http://localhost:8050` . It is **read-only** and reloads `meter_points.yaml` live, so it auto-discovers terminals you add to the YAML (unlike the CLI report path).

- On load / "Reload Data", `reload_data()` pulls `flows` , `eia_weekly` , and `ais_daily_inference` into client-side stores and builds the gas-day dropdown (union of days with flow data and days with AIS data).
- `update_dashboard()` re-renders on any control change: KPI cards (total visibility + per-region), a cross-region time series with the EIA overlay, and per-tab (U.S. LNG / Mexico / Canada) charts — terminal bars, % of nameplate, per-terminal stack, pipeline breakdown — plus the raw-flows and AIS tables.
- **Terminal categorization** is by name prefix (`Mexico - ...` , `Canada - ...` , else U.S. LNG). `terminal_totals()` sums `direction='delivery'` for exports but `direction='receipt'` for the Canada *import* points (Sumas, Waddington, Emerson), so an import shows as a positive number.
- The green/amber/gray **confidence outlines** encode data quality physically: "exact" = a single metered border crossing (the whole flow), "lower bound" = a multi-fed LNG terminal where private feeders are invisible, "not captured" = no scraper yet.

2.6 Scheduled automation (Windows Task Scheduler)

- `tools/daily_pull.ps1` — runs `pull --pipeline all --date yesterday --cycle evening` nightly, appending stdout+stderr to `data/logs/daily_pull.log`. "Yesterday evening" because the evening cycle (~9 PM CPT) is the refined next-day forecast view.
- `tools/ais_track.ps1` — runs `ais-collect --duration 600` then `ais-infer --date today`, hourly, logging to `data/logs/ais_track.log`.

Both scripts hardcode the Python interpreter path and set `PYTHONPATH=src`.

2.7 Nomination cycles — the time dimension you must understand

Pipelines re-post scheduled quantities several times per gas day as nominations firm up. The project's canonical cycle names (`models.py`: `timely`, `evening`, `intraday1-3`, `confirmed`) are mapped to each portal's own naming by per-scraper tables (`CYCLE_TO_KMI`, `CYCLE_TO_WILLIAMS`, `CYCLE_TO_TETCO_PREFIX`, `CYCLE_TO_IROQUOIS`, ...):

- **timely** (~10 AM CPT) — first day-ahead look.
- **evening** (~9 PM CPT) — refined day-ahead view; the default and what's used for forecasting.
- **confirmed** — maps to each portal's "best available / post" final number; use for historical actuals.

Caveat: several portals (`energytransfer`, `et_ipost`, `williams_nwp`, `tceconnects`, `iroquois`) only return their **most-recent posted snapshot** and ignore the requested date/cycle. Only `kmi`, `enbridge`, and `williams` honor an arbitrary gas-day. This limits historical backfill — see §8.

3. Repository layout

NG	Production/	← repo root (git)
	README.md	⚠ stale (LNG-only POC description)
	requirements.txt	deps: requests, pandas, click, playwright, dash, websockets...
	dashboard.py	Plotly Dash app – the modern, dynamic UI (reads YAML live)
	NG Production/	
	└─ LNG_Feedgas_Template.md	the source-of-truth workflow this whole repo automates
	docs/	
	└─ portal_notes.md	⚠ stale – per-portal scraping recipes & coverage gaps
	data/	
	└─ feedgas.db	SQLite – all history (flows + eia + ais tables)
	└─ exports/	generated Part-4 CSV + Part-5 prompt .txt
	└─ cenagas/, *.pdf	Mexico CENAGAS PDFs (manual backfill source)
	tools/	one-off research/ops scripts (NOT part of the package)
	└─ daily_pull.ps1	Task Scheduler: nightly `pull` → data/logs/
	└─ ais_track.ps1	Task Scheduler: hourly AIS collect+infer
	└─ discover_kmi.py / discover_tco.py / discover_cp.py	meter-ID discovery
	└─ probe_iroquois*.py / dump_iroquois.py / probe_cenagas.py / cenagas_pdf_backfill.py	
	src/ng_feedgas/	the installable package (run via PYTHONPATH=src)
	└─ __main__.py / cli.py	click CLI: pull / report / show / eia-fetch / ais-collect / ais-infer
	└─ models.py	FlowRecord dataclass; Cycle/Direction literals; error types
	└─ validators.py	Part-6 reasonableness checks (nameplate %, US-total band)
	└─ config/	
	└─ └─ __init__.py	load_config() → Config + MeterPoint dataclasses
	└─ └─ meter_points.yaml	★ THE HEART – terminal→pipeline→meter map + bounds
	└─ scrapers/	
	└─ └─ base.py	BaseScraper(ABC) + ScrapeContext
	└─ └─ __init__.py	SCRAPERS registry {name: class}
	└─ └─ kmi.py	★ canonical: viewstate POST → xlsx (NGPL/EEC/TGP/SNG/EPNG/Sierrita)
	└─ └─ tcenergy.py	delegates to KMI (code=TGP)
	└─ └─ enbridge.py	TETCO: ASP.NET __doPostBack → CSV
	└─ └─ williams.py	Transco: 4-step JSP flow → HTML regex
	└─ └─ williams_nwp.py	Northwest Pipeline: Struts → HTML regex (Sumas import)
	└─ └─ energytransfer.py	et_tgc legacy: direct CSV URL (Trunkline only)
	└─ └─ et_ipost.py	generalized ET: asset= param CSV (TGC/TW/FEP/PEPL/FGT)
	└─ └─ tceconnects.py	Playwright: SSRS export (TCO/CGT/ANR)
	└─ └─ iroquois.py	Playwright: ExtJS SPA behind Imperva (Waddington)
	└─ storage/	
	└─ └─ schema.sql	flows table (UNIQUE gas_day,cycle,pipeline,meter_point)
	└─ └─ db.py	connect/upsert/terminal_totals/us_totals_range
	└─ └─ export.py	⚠ Part-4 CSV/text – hardcoded 8-LNG TERMINAL_ORDER
	└─ analysis/	
	└─ └─ stats.py	DailyStats: DoD, 7d, 30d averages
	└─ └─ prompt.py	⚠ Part-5 prompt – also uses the 8-LNG TERMINAL_ORDER
	└─ calibration/	
	└─ └─ schema.sql	eia_weekly + ais_observations/ships/daily_inference
	└─ └─ eia.py	EIA v2 API → monthly US LNG export totals (Bcf/d)
	└─ └─ ais.py	aisstream.io WebSocket → berth occupancy → crude estimate

4. The data model

FlowRecord (src/ng_feedgas/models.py) — the universal currency: gas_day, cycle, terminal, pipeline, meter_point, mmcfd, direction, source_url, scraped_at

SQLite flows table (`src/ng_feedgas/storage/schema.sql`): one row per (`gas_day`, `cycle`, `pipeline`, `meter_point`) — that 4-tuple is the upsert key, so re-pulling a day overwrites cleanly. `terminal_totals()` sums `direction='delivery'` per terminal.

Units: every portal posts in Dth/d ($\approx$ MMBtu/d). Convention treats Dth $\approx$ Mcf for feedgas, so every scraper divides by 1000 $\rightarrow$ **MMcf/d**.

Calibration tables (`src/ng_feedgas/calibration/schema.sql`): `eia_weekly` (now actually monthly), `ais_observations`, `ais_ships` (MMSI $\rightarrow$ type registry), `ais_daily_inference` .

5. The config-driven design — `meter_points.yaml` is everything

`src/ng_feedgas/config/meter_points.yaml` defines every terminal, its nameplate, and which (`scraper`, `pipeline_code`, `meter_id`) feeds it. The CLI builds scrapers from it; the dashboard reads it live for its terminal catalog, ordering, and nameplates. **Add a terminal here and the dashboard picks it up automatically** (the CLI report path does *not* — see §8).

Three terminal categories, distinguished by name prefix in the dashboard: `Mexico - ...`, `Canada - ...`, else U.S. LNG.

6. How to run it

```
# setup
python -m venv .venv; .venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m playwright install chromium          # needed for tceconnects + iroquois

$env:PYTHONPATH = "src"                      # package isn't pip-installed; run via PYTHONPATH

# daily pull (all scrapers, yesterday evening)  $\rightarrow$  SQLite + Part-6 warnings to stderr
python -m ng_feedgas pull --date yesterday --cycle evening --pipeline all
python -m ng_feedgas pull --pipeline kmi      # one scraper

python -m ng_feedgas show --date yesterday --cycle evening    # print Part-4 table
python -m ng_feedgas report --date yesterday --cycle evening  # write CSV + prompt

# optional calibration (need free API keys via env vars)
python -m ng_feedgas eia-fetch --weeks 52      # EIA_API_KEY
python -m ng_feedgas ais-collect --duration 300 # AISSTREAM_API_KEY
python -m ng_feedgas ais-infer --date today

python dashboard.py                                # http://localhost:8050
```

Automation is via **Windows Task Scheduler** driving `tools/daily_pull.ps1` (nightly) and `tools/ais_track.ps1` (hourly). Note both hardcode `C:\Users\Intern\...\Python312\python.exe` .

7. How to extend it

Add a new meter to an existing pipeline: add a `feeds:` entry under the terminal in YAML. Zero code.

Add a new terminal/border point: add a `terminals:` block in YAML. Dashboard updates automatically.

Add a new pipeline/portal (new scraper):

1. Create `scrapers/yourpipe.py` with a class extending `BaseScraper`, implementing `fetch(ctx) → list[FlowRecord]`. Copy the closest existing scraper as a template: - **Direct CSV** → `energytransfer.py` (simplest) - **ASP.NET viewstate POST** → `file` → `kmi.py` or `enbridge.py` - **HTML report regex** → `williams.py` / `williams_nwp.py` - **JS-only / bot-protected** → `tceconnects.py` / `iroquois.py` (Playwright)
2. Register it in `scrapers/__init__.py` `SCRAPERS` dict, and add its name to the `--pipeline` choices in `cli.py`.
3. Add the `scraper:` key to the relevant YAML feeds.
4. Use a `tools/discover_*.py` script to find the meter IDs (that's exactly what they're for).

Every scraper follows the same internal shape: `_download/_fetch_rows` → `_match_meters` (by `meter_id` first, then `location_name` substring) → `_parse_number` → divide by 1000 → `FlowRecord`. Failures are isolated per-scraper in the CLI loop, so one broken portal won't abort the run.

8. Known gaps & gotchas (read before building)

- **CLI report path is stale vs. the dashboard.** `export.py` and `prompt.py` use a **hardcoded 8-LNG** `TERMINAL_ORDER`. So `report / show` / the Part-5 prompt **silently drop all Mexico and Canada points**. The dashboard reads YAML dynamically and shows everything. If you touch the CSV/prompt outputs, this is the first thing to fix (make `TERMINAL_ORDER` derive from config like the dashboard does).
- **"Latest snapshot" scrapers ignore the date.** `energytransfer`, `et_ipost`, `williams_nwp`, `tceconnects`, `iroquois` return the most-recent posted snapshot regardless of `--date`. Only `kmi` / `enbridge` / `williams` honor a gas-day. Historical backfill is limited.
- **No tests, no CI, no packaging.** No `pyproject.toml` / `setup.py`; everything runs via `PYTHONPATH=src`. There's no test suite at all — a good early investment.
- **Dashboard `signed_mmcfd` doesn't actually net.** It returns receipts and deliveries both positive despite "net" language; Canada import/export handling is done by terminal-name allowlist in `_is_canada_import`.
- **DB has only the evening cycle and 3 gas days** (2026-05-21, -26, -27). It's early-stage data.
- **Missing scrapers** (documented in YAML comments): Golden Pass (gasnom.com ColdFusion), Empire/Chippawa, Emerson (Viking/Great Lakes/Northern Border), Gulf South, Roadrunner/ONEOK, and Texas-intrastate Mexico crossings (only reachable via CENAGAS PDFs).
- **Coverage is a lower bound for LNG.** Multi-fed terminals (Sabine, etc.) miss private/intrastate feeders (e.g. Cheniere's Creole Trail). The dashboard encodes this honestly as a green/amber/gray "confidence"

outline — border crossings are "exact" (single metered pipe), LNG terminals are "lower bound."

9. Scraper reference

Scraper (<code>--pipeline</code>)	Portal	Pipelines	Tech
<code>kmi</code>	pipeline2.kindermorgan.com	NGPL, EEC, TGP, SNG, EPNG, Sierrita (via <code>code=</code>)	requests + viewstate POST + xlsx
<code>tcenergy</code>	(delegates to KMI, code=TGP)	TGP	—
<code>enbridge</code>	rtba.enbridge.com	TETCO	requests + <code>__doPostBack</code> + CSV
<code>williams</code>	1line.williams.com	Transco	requests + 4-step JSP + HTML regex
<code>williams_nwp</code>	northwest.williams.com	Northwest Pipeline	requests + Struts + HTML regex
<code>et_tgc</code>	tgcmessage.energytransfer.com	Trunkline (legacy)	requests + direct CSV URL
<code>et_ipost</code>	twtransfer.energytransfer.com	TGC, TW, FEP, PEPL, FGT	requests + <code>asset=</code> CSV
<code>tceconnects</code>	ebb.tceconnects.com	TCO, Columbia Gulf, ANR	Playwright (SSRS, JS-only)
<code>iroquois</code>	iol.iroquois.com	Iroquois	Playwright (ExtJS SPA, Imperva)

Playwright scrapers require: `pip install playwright && python -m playwright install chromium`.

10. Coverage map (from `meter_points.yaml`)

U.S. LNG terminals: Sabine Pass, Corpus Christi, Freeport, Cameron, Cove Point, Elba Island, Plaquemines (active feeds); Calcasieu Pass + Golden Pass (no public EBB / no scraper yet).

Mexico exports (KMI EPNG/Sierrita): Sasabe, North Baja, Samalayuca, Cananea, Willcox, El Fresnal, Mendoza Trail.

Canada crossings: Niagara (TGP delivery), Sumas (NWP receipt), Waddington (Iroquois); Chippawa (Empire) + Emerson (Viking/Great Lakes/Northern Border) not yet scraped.

US-total reasonableness band (validators): **10,000–30,000 MMcf/d** (LNG + Mexico + Canada combined).

11. Suggested first moves

1. Make `TERMINAL_ORDER` config-derived so CLI exports match the dashboard (highest-value, low-risk).
2. Refresh `README.md` + `docs/portal_notes.md` to the current LNG+Mexico+Canada scope.
3. Add a minimal `pyproject.toml` + a few `pytest` tests around `_match_meters` / `_parse_number` / `upsert_flows` (pure functions, easy wins).
4. Pick one missing high-volume scraper — **Golden Pass** (~2.5 Bcf/d) is the biggest single gap.